czii-cryo-et-object-identification

8th Place Solution for the CZII - CryoET Object Identification Competition + Code Released

Rank	8
Team	I Cryo Everyteim
Author	Sergio Alvarez
Topic ID	561515
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561515
Retrieved with Kaggle CLI on 2026-10-21.

First, we thank the competition host and Kaggle staff for organizing this competition. Below, we introduce the solution of the team I Cryo Everyteim -- @sirapoabchaikunsaeng, @iamparadox, @itsuki9180, @sersasj --

Context

- Business context: competition overview
- Data context: competition data

Overview of the approach

Our final submissions consisted of four 3D U-Net model soups, trained with different model sizes, parameters, and training data to ensure strong model complementarity. All 3D U-Nets were trained using patch sizes of (128, 128, 128) but were inferred with patch sizes of (160, 384, 384), with a 25% overlap using gaussian reconstruction to handle border artifacts. Additionally, we employed geometric test-time augmentation (TTA), including flipping and transpose.

Models

Our models were originally based on the host's example notebook and @fnands notebook . We utilized 3D U-Net architectures from the MONAI library, trained with patch sizes of (128, 128, 128).
The models were 3 levels deep with strides of (2, 2, 1). We started with a simple model:

```
:
:
:
```

This single model with our inference strategy reached 0.744 in public leaderboard score.
Later, we trained more complex U-Nets with configs:

```
: ,
: ,
: ,
```

The model with above config alone achieved a score of 0.759 LB.

```
: ,
: ,
: ,
```

We also applied a dropout of 0.2 or 0.3 in the model.

Training

The models were pre-trained on six synthetic tomograms denoised with Gaussian denoising—specifically, the 'TS_0', 'TS_1', 'TS_10', 'TS_11', 'TS_12', and 'TS_13' tomograms.
@sersasj note: Gaussian denoising was applied because it visually improved the WBP tomograms particle visualization and was easy to implement. I hypothesized that better results could be achieved with a more

advanced denoiser, but attempting to code a model for denoising used too much of my Kaggle quota, so I gave up.
Pretraining not only reduced the time needed for the models to learn the particles but also increased the LB score by roughly 0.01.
Both pretraining on synthetic data and fine-tuning uses the following transformations:

```
Compose([
   RandCropByLabelClassesd(
       keys=[, ],
       =,
       spatial_size=[128, 128, 128],
       =7,  # background,  all 6 classes
       =16,
   ),
   RandFlipd(keys=[, ], =0.5, =0),
   RandFlipd(keys=[, ], =0.5, =1),
   RandFlipd(keys=[, ], =0.5, =2),
])
```

We used MONAI's DiceCELoss and optimized the models with AdamW, employing a learning rate reduction on plateau and an initial learning rate of 1e-3.
We also experimented with Exponential Moving Average (EMA), which showed good results. In the last 3 days of the competition, we trained models using other tomo types: "denoised", "ctfdeconvolved", "isonetcorrected". For almost the entire competition, we hadn't found any increase in LB strategies other than geometric data augmentation in training (flip and rotate); nevertheless, training with other tomo types yielded good results.

Validation strategy

We primarily relied on out-of-fold predictions from our k-fold models. Specifically, we implemented a 7-fold cross-validation approach where we trained on all tomographies except one, which was used as a validation set. This process was repeated seven times, with each fold serving as the validation set once. This helped us obtain a score that correlated well with the public leaderboard score, typically 0.01 to 0.02 points higher.
One limitation we encountered was our inability to apply this validation strategy when testing ensembles of our model soups. In these cases, we had to rely on leaked data scores. While this approach didn't correlate as strongly with leaderboard scores, most of the time it allowed us to rank the performance of different ensembles, and compare TTA strategies.

Details of the submission

Inference

During inference, we use MONAI's sliding_window_inference with 25% overlap and Gaussian reconstruction to handle border predictions more accurately; this boosted the public LB by around 0.002. **One of the greatest findings was making the inference in patches as large as possible; we used a window size of (160, 384, 384), boosting our score by around 0.01 compared to a size of (128, 128, 128).** (Thanks @iamparadox for finding this)
We also experimented with combining different sizes of predictions, for example (160, 384, 384) + (136, 248, 248), but inference took much longer while yielding inconclusive gains. Therefore, we abandoned this

approach in favor of creating more ensembles and employing more TTA.
For post-processing, we improved particle detection by combining model predictions at the logits level rather than averaging probabilities directly.
We then refined particle identification using watershed segmentation, which helped separate touching or overlapping particles. The watershed process created binary masks using particle-specific certainty thresholds, computed distance transforms to measure particle separation, identified distinct particles using local maxima as markers, and finally applied watershed segmentation to determine particle boundaries. We switched from using skimage to cucim and cupy to decrease the inference time.
We also used specific blob threshold sizes for each particle to decrease the number of false positives in our predictions.
Moreover, we used flip along the X and Y axes and transpose TTA.

Ensembles

Ensembling was always one of the main ideas in our team. When the team was fully formed 2 weeks before the deadline, we were all around a 0.73 LB score, around 50th place on the leaderboard. At the time, I (@sersasj) had a YOLO scoring 0.682 and a U-Net scoring 0.692 that, when ensembled, reached 0.732 in LB. The rest of the team had only a 3D U-Net that reached 0.739. So it was obvious that we would ensemble everything and achieve amazing results; sadly, it didn't work.
Nevertheless, we learned valuable lessons from this experience. My 3D U-Net was ~30 times smaller than the 0.739 scoring U-Net, but it proved quite effective when used with the team's amazing inference techniques: larger patches, blob thresholds, ensemble at logits level, and watershed processing. After implementing pretraining and additional training, it achieved a 0.744 leaderboard score.
Thanks to our group work (especially @sirapoabchaikunsaeng and @iamparadox), we successfully trained several U-Net variants derived from the 0.744 model by modifying parameters, implementing EMA (Exponential Moving Average), incorporating more data, and applying the techniques mentioned above.
By doing so, we had various U-Nets that were complementary to each other, but we couldn't use them all. To manage "using" all we had, @itsuki9180 presented the idea and code for a technique called model soup, which essentially averages the weights of multiple models from the same pre-trained weights.
We applied this to the models from our k-fold training.
Our final ensemble consisted of:

1. A soup of 7 folds trained with parameters:
    - Channels: (32, 64, 128, 128)
    - Number of residual units: 1
    - Pretrained on synthetic data
    - Trained on denoised images without EMA
2. A soup of 7 folds trained with parameters:
    - Channels: (32, 64, 128, 256)
    - Number of residual units: 2
    - Pretrained on synthetic data
    - Trained on all denoised, IsoNet-corrected, and CTF-deconvolved images without EMA
3. A soup of 3 folds (TS_69_2, TS_86_3, TS_99_9) trained with parameters:
    - Channels: (32, 96, 256, 384)
    - Number of residual units: 2

    - Pretrained on synthetic data
    - Trained on denoised images with EMA
4. A soup of 7 folds trained with parameters:
    - Channels: (32, 96, 256, 384)
    - Number of residual units: 2
    - Pretrained on synthetic data
    - Trained on all denoised, IsoNet-corrected, and CTF-deconvolved images without EMA

What didn't work:

- Increased Augmentations
- CutMix, MixUp
- 2D U-Net approaches (reached 0.636 lb only)
- Ensembling with YOLO (for final solution, works well to reach ~0.73 lb)
- Bigger and deeper U-Nets
- Cross entropy only loss functions

Sources section

- Model Soup paper
- Host's example notebook
- @fnands notebook

Code

8th place solution of kaggle czii competition code github
Submission Notebook
Submission Notebook Clean Version

Supplemental Q&A

> **Ma Edward · 2025-02-06T14:57:47.563000**
>
> Congratulations!!! Very insightful ensemble solutions.
> Would you mind sharing the single model performance in the final ensemble models?

> **IAmParadox · 2025-02-06T15:00:07.970000**
>
> one of the single model of final ensemble was of following config:
>
> `"channels": (32, 64, 128, 256), "strides": (2, 2, 1), "num_res_units": 2,`
>
> we created model soup from all the folds and it's public LB was 0.762

Aurora_blue · 2025-02-07T16:22:55.983000

Congratulations!

@iamparadox
This is interesting for me.

> One of the greatest findings was making the inference in patches as large as possible; we used a window size of (160, 384, 384), boosting our score by around 0.01 compared to a size of (128, 128, 128).

Do you train model with same patch size or not ?
If not, I cannot understand why this improve score.
I would appreciate it if you could share any theoretical background.

IAmParadox · 2025-02-08T14:16:10.763000

The model was only trained on 128x128x128 patch size but it was inferred on 160x384x384 patch size. I honestly don't know a lot of theoretical background on this topic, but the idea here's that the contextual information doesn't change while we are doing the inference, in a sense that suppose, a 3x3x3 conv filter expects high value in the center pixel of the input image to activate then this doesn't change even if I increase the patch size at inference and the second reason is doing inference on larger patchsize reduces the border artifacts.

The large patchsize trick will not work if we downsample the image into smaller dimension, and then infer them on higher dimension because in this case the contextual information will change, i.e it will see finer details that it has never seen before.

Sergio Alvarez · 2025-02-09T00:18:31.917000

@clearwaterkzk Just to shown the difference bigger patches made, I submitted our best submission with patches 128.
Best submission: public lb: 0.78022, private lb: 0.77612
Patches 128 submission: public lb: 0.77464, private lb: 0.76604